

Ethical Hacking & Vulnerability Assessment Report

Project: Secure Node.js Application (Week 5) **Date:** June 9, 2026

Executive Summary

This report outlines the ethical hacking and vulnerability assessment conducted on the target Node.js application. The primary objective was to identify, exploit, and patch critical vulnerabilities, specifically focusing on SQL Injection (SQLi) and Cross-Site Request Forgery (CSRF), ensuring the application adheres to standard security practices.

1. SQL Injection (SQLi) Vulnerability & Remediation

1.1 Vulnerability Discovery

During the reconnaissance phase, an insecure SQLite endpoint (`/search`) was identified. The endpoint was directly concatenating user input into the SQL query without proper sanitization, making it highly vulnerable to SQL Injection attacks.

1.2 Exploitation (Proof of Concept)

A manual SQL Injection attack was launched using the payload `' OR '1'='1`. This successfully altered the query logic to always evaluate to true, resulting in the unauthorized extraction of the entire database contents (including hidden products).

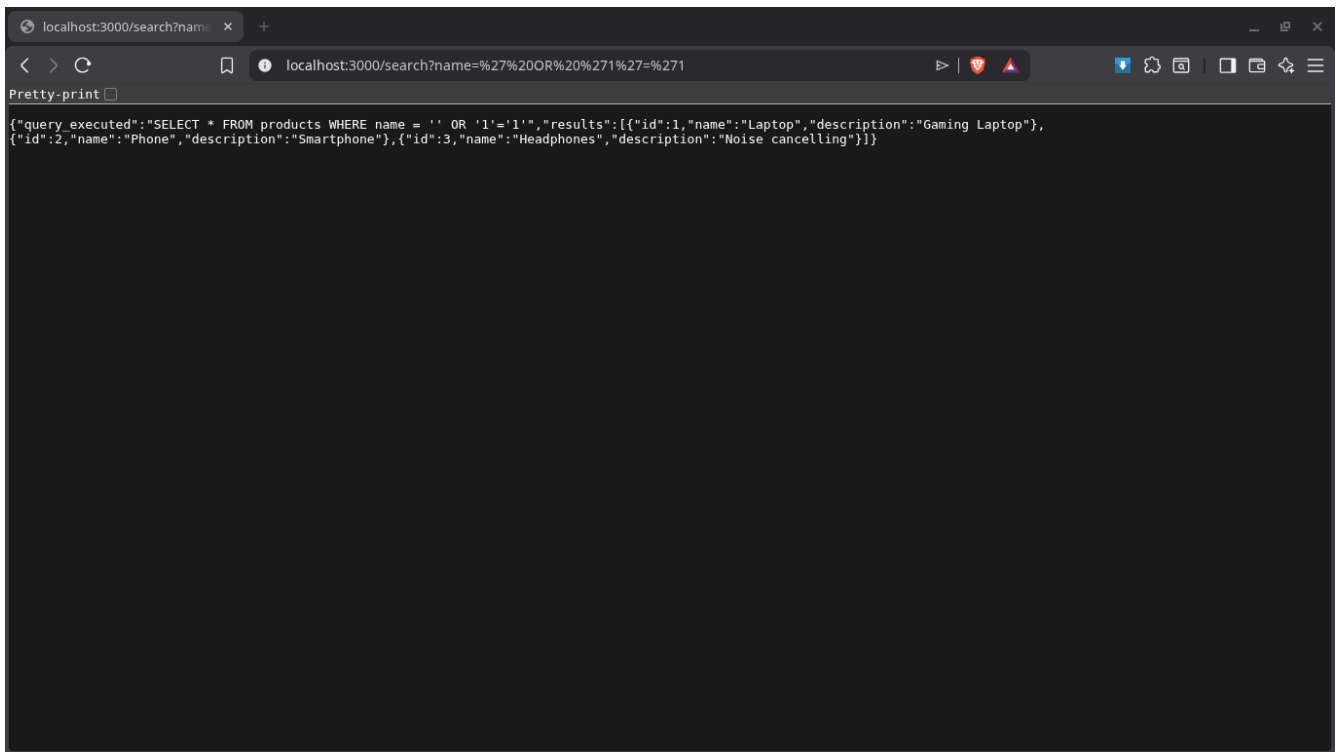


Figure 1: Successful SQL Injection attack revealing entire database contents.

1.3 Remediation & Patching

To permanently secure the application against SQLi, the vulnerable string concatenation was replaced with **Prepared Statements (Parameterized Queries)**. This ensures that the database engine treats user input strictly as data, not as executable code.

2. Cross-Site Request Forgery (CSRF) & Traffic Interception

2.1 Traffic Analysis via Burp Suite

To simulate an external attacker's perspective, traffic between the client and the Node.js server was routed through a Kali Linux Virtual Machine. **Burp Suite** was utilized to intercept and analyze the HTTP requests, specifically targeting state-changing endpoints like the `/admin` panel.

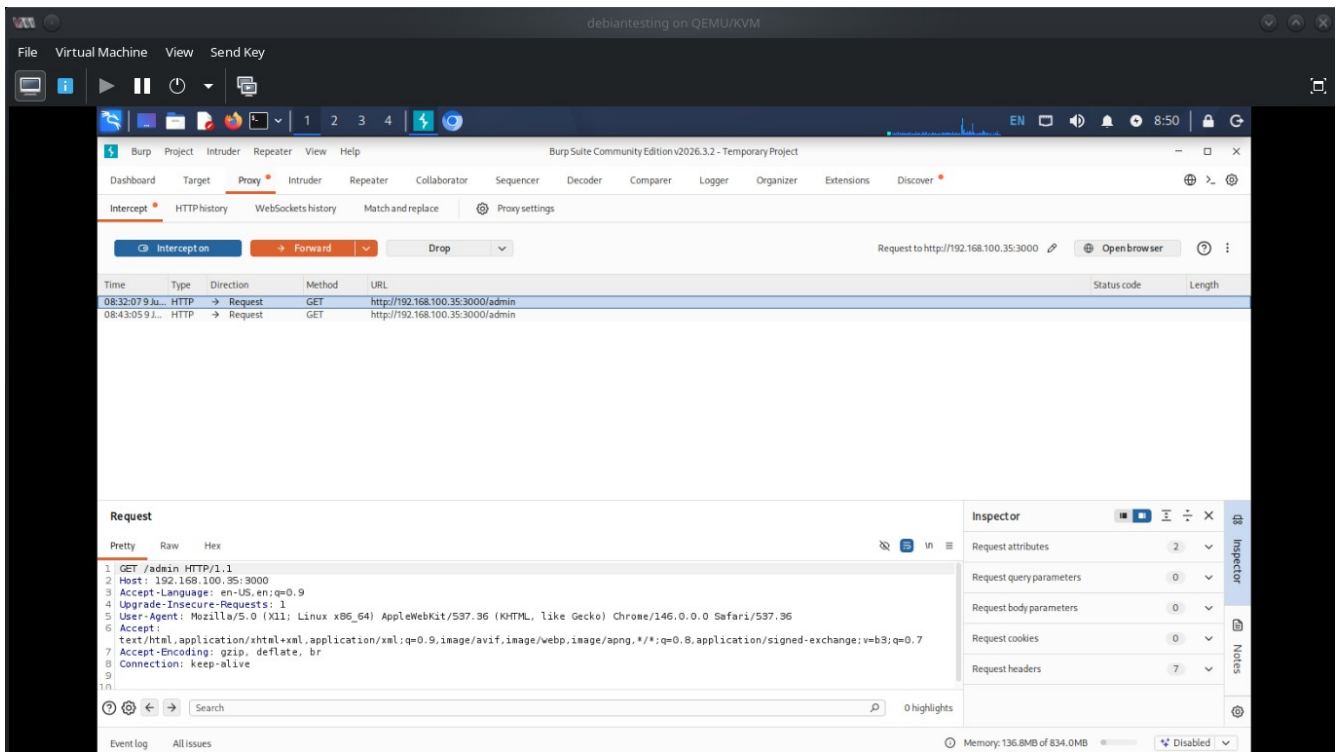


Figure 2: Intercepting application traffic using Burp Suite proxy.

2.2 Vulnerability & Remediation

Initial analysis showed that state-changing requests lacked validation to verify their origin. To mitigate this, the `csrf` middleware was integrated into the Node.js backend. This established a robust CSRF protection mechanism by generating unique tokens for client sessions.

2.3 Defense Verification

An automated test using `curl` was executed to simulate an unauthorized POST request lacking a valid CSRF token. The backend security successfully detected the anomaly and blocked the request, returning a `403 Forbidden` status.

```
Vulnerable-nodejs: bash — Konsole
New Tab Split View Copy Paste Find
[morbiuss@rust Vulnerable-nodejs]$ curl -X POST http://localhost:3000/admin
<!DOCTYPE html><html><head><title></title><link rel="stylesheet" href="/stylesheets/style.css"><link rel="stylesheet" href="/stylesheets/tooltip.css"></head><body><h1>
Invalid csrf token</h1><h2>403</h2><pre>ForbiddenError: invalid csrf token
    at csrf (/home/morbiuss/Documents/Internship/Cybersecurity-Internship-Tasks/Vulnerable-nodejs/node_modules/express/lib/router/index.js:112:19)
    at Layer.handle [as handle_request] (/home/morbiuss/Documents/Internship/Cybersecurity-Internship-Tasks/Vulnerable-nodejs/node_modules/express/lib/router/layer.js:
95:5)
    at trim_prefix (/home/morbiuss/Documents/Internship/Cybersecurity-Internship-Tasks/Vulnerable-nodejs/node_modules/express/lib/router/index.js:317:13)
    at /home/morbiuss/Documents/Internship/Cybersecurity-Internship-Tasks/Vulnerable-nodejs/node_modules/express/lib/router/index.js:284:7
    at router.process_params (/home/morbiuss/Documents/Internship/Cybersecurity-Internship-Tasks/Vulnerable-nodejs/node_modules/express/lib/router/index.js:335:12)
    at next (/home/morbiuss/Documents/Internship/Cybersecurity-Internship-Tasks/Vulnerable-nodejs/node_modules/express/lib/router/index.js:275:10)
    at serveStatic (/home/morbiuss/Documents/Internship/Cybersecurity-Internship-Tasks/Vulnerable-nodejs/node_modules/serve-static/index.js:75:16)
    at Layer.handle [as handle_request] (/home/morbiuss/Documents/Internship/Cybersecurity-Internship-Tasks/Vulnerable-nodejs/node_modules/express/lib/router/layer.js:
95:5)
    at trim_prefix (/home/morbiuss/Documents/Internship/Cybersecurity-Internship-Tasks/Vulnerable-nodejs/node_modules/express/lib/router/index.js:317:13)
    at /home/morbiuss/Documents/Internship/Cybersecurity-Internship-Tasks/Vulnerable-nodejs/node_modules/express/lib/router/index.js:284:7</pre><script src="http://aja
x.googleapis.com/ajax/libs/jquery/2.1.1/jquery.min.js"></script><script src="/javascripts/global.js"></script></body></html>[morbiuss@rust Vulnerable-nodejs]$
```

Figure 3: Server successfully blocking an unauthorized request with an invalid CSRF token.

Conclusion

The application has been successfully tested and fortified against major OWASP Top 10 vulnerabilities. By implementing Parameterized Queries and CSRF token validation, the attack surface has been significantly minimized, ensuring a secure environment for end-users.